

StudyRoom TechLab Billing

System Workflow and Add-on Developer Guide

This guide explains how the current Laravel billing system works, how company subscription add-ons become visible in the sidebar, and how to add future modules such as SMS or M-Pesa without breaking the existing app.

Area	Current Design
Framework	Laravel with authenticated Inertia pages plus Blade ISP pages where needed
Brand	StudyRoom TechLab Billing / StudyRoom ISP Billing
Add-ons	StudyRoom packages live in packages/studyroomtechlab and are loaded beside existing packages
Access	PlanModuleCheck controls whether a company can see and open subscribed add-ons

1. How the System Works

The application has a platform layer and company layer. Super Admin manages global settings, add-ons, subscription plans, users, and the installation or update flow. Company users subscribe to a plan. The plan determines which modules are available to that company.

StudyRoom TechLab add-ons are installed like first-party modules. Each add-on can register routes, controllers, menus, translations, migrations, and frontend pages. When the add-on is enabled and assigned through a subscription plan, the company sees it in the sidebar.

WiFi Billing is the ISP module. It provides ISP billing navigation, internet packages, customers, MikroTik routers, and provisioning routes. WiFi Billing HRM is the field-team companion module for installers, jobs, and support workflows.

Important Login Flow

- User signs in through the existing authentication system.
- The sidebar reads the user type, active plan, and active modules.
- If the user has a subscribed add-on, its menu definition is merged into the main sidebar.
- Routes use auth, verified, and PlanModuleCheck middleware so unsubscribed users cannot open the module by URL.

Core Tables Used by the Add-on System

Table	Purpose
add_ons	Stores installed modules, version, enabled state, and admin visibility.
plans	Stores subscription plans and the add-ons/features included in each plan.
users	Stores user type, company ownership, active plan, and active modules.
isps	Stores ISP/company records and assigned ISP admin user.
customers, internet_packages, mikrotik_routers	Stores the operational ISP billing data used by WiFi Billing.

2. Current StudyRoom Add-ons

Add-on	Route	Status
WiFi Billing	/wifi-billing/dashboard plus ISP package, customer, and router pages	Active foundation module
WiFi Billing HRM	/wifi-billing/hrm/dashboard	Sidebar dashboard added; detailed HRM workflow is next phase
Landing Page	/landing-page or package-defined landing routes	Rebranded StudyRoom visible text

Minimum Package Structure

Each new StudyRoom add-on should follow this structure:

```
packages/studyroomtechlab/ModuleName
packages/studyroomtechlab/ModuleName/module.json
packages/studyroomtechlab/ModuleName/src/Providers/ModuleNameServiceProvider.php
packages/studyroomtechlab/ModuleName/src/Routes/web.php
packages/studyroomtechlab/ModuleName/src/Http/Controllers
packages/studyroomtechlab/ModuleName/src/Resources/js/menus/company-menu.ts
packages/studyroomtechlab/ModuleName/src/Resources/js/Pages
```

Menu Visibility Checklist

- The add-on exists in the add_ons table and is enabled.
- The subscription plan includes the add-on name exactly as the package uses it.
- The company user has the plan assigned and active modules synced.
- The package exports a company menu file and the asset build has been run.
- The route uses PlanModuleCheck with the same module key.

3. How to Add a New Add-on

Use this process for future modules such as SMS, M-Pesa, inventory, ticketing, or customer portal features.

Step	Action	Details
1	Create the package folder	Create packages/studyroomtechlab/SmsModule or packages/studyroomtechlab/MpesaPayments.
2	Add module metadata	Create module.json with name, alias, version, description, and provider class.
3	Create a service provider	Register routes, migrations, views, translations, and frontend resources.
4	Create protected routes	Use auth, verified, and PlanModuleCheck:ModuleName.
5	Create controller and page	Add a dashboard page first. Keep the first phase small and verify access.
6	Create menu export	Add company-menu.ts so the sidebar can discover the module.
7	Register add-on	Seed or install the add-on into the add_ons table.
8	Assign to plan	Super Admin adds the module to a subscription plan, then company subscribes.
9	Build assets and clear cache	Run npm build and Laravel cache clear commands.

4. Example: SMS Module

An SMS module should start as a small dashboard with provider settings. Do not build every feature at once. First make the module installable, visible in the sidebar, and protected by subscription.

Suggested Features

- SMS provider settings: provider name, API key, sender ID, test phone.
- Message templates: payment reminder, account activated, account suspended, installation booked.
- Send log: customer, message, status, cost, provider response, sent_at.
- Queue support: send through Laravel jobs later instead of blocking the page.

Suggested Tables

```
sms_settings, sms_templates, sms_messages
```

First Route Set

```
GET /sms/dashboard
GET /sms/settings
POST /sms/settings
POST /sms/test
```

Safe First Implementation

Build settings and a test-send placeholder first. Add real provider API calls only after settings, permissions, and logs work. This keeps debugging simple and avoids charging SMS credits accidentally during development.

5. Example: M-Pesa Module

An M-Pesa module should be treated as a payment add-on. Keep credentials, callback routes, transaction logs, and invoice/customer matching separate from WiFi Billing core logic.

Suggested Features

- M-Pesa settings: environment, shortcode, passkey, consumer key, consumer secret, callback URL.
- STK Push request page for testing customer payments.
- Public callback route outside auth middleware for Safaricom callbacks.
- Transaction log with checkout request ID, phone, amount, result code, and raw callback JSON.
- Optional automatic customer billing status update after successful payment.

Suggested Routes

```
GET /mpesa/dashboard
GET /mpesa/settings
POST /mpesa/settings
POST /mpesa/stk-push
POST /mpesa/callback (public route, no auth)
```

Important Safety Notes

- Never store secrets in frontend JavaScript.
- Keep callbacks public but validate payloads and log every request.
- Use sandbox credentials first. Move to production only after callback and reconciliation are tested.

6. Commands and Troubleshooting

Useful Commands

```
C:\xampp\php\php.exe artisan optimize:clear  
C:\xampp\php\php.exe artisan config:clear  
C:\xampp\php\php.exe artisan route:list  
npm run build
```

Common Problems

Problem	Likely Cause	Fix
Add-on visible in plan but missing from sidebar	Menu file is missing, package resources not discovered, or frontend assets not rebuilt.	Add company-menu.ts, verify active module key, then run npm run build.
403 after clicking add-on	User plan or active modules does not contain the module key, or ISP ownership check failed.	Assign module to plan, resubscribe/sync company, and check users.active_plan plus user_active_modules.
Route not defined	Package route file did not load or named route is different from menu href.	Run route:list and match the route name used by the menu.
Page not found in Inertia	Render name does not match the compiled Pages path.	Match Inertia render name to Resources/js/Pages file path.
Installer or updater repeats migrations	Existing table was imported manually but migration is still pending.	Make migrations table match database state or make migration idempotent with Schema checks.

Recommended Next Phases

- Phase 2: Complete WiFi Billing operational pages and real MikroTik connection testing.
- Phase 3: Build WiFi Billing HRM installer and support-ticket workflow.
- Phase 4: Add SMS notifications as a separate add-on.
- Phase 5: Add M-Pesa payments as a separate payment add-on with public callback route.